AKADEMIA HANDLOWA NAUK STOSOWANYCH W RADOMIU

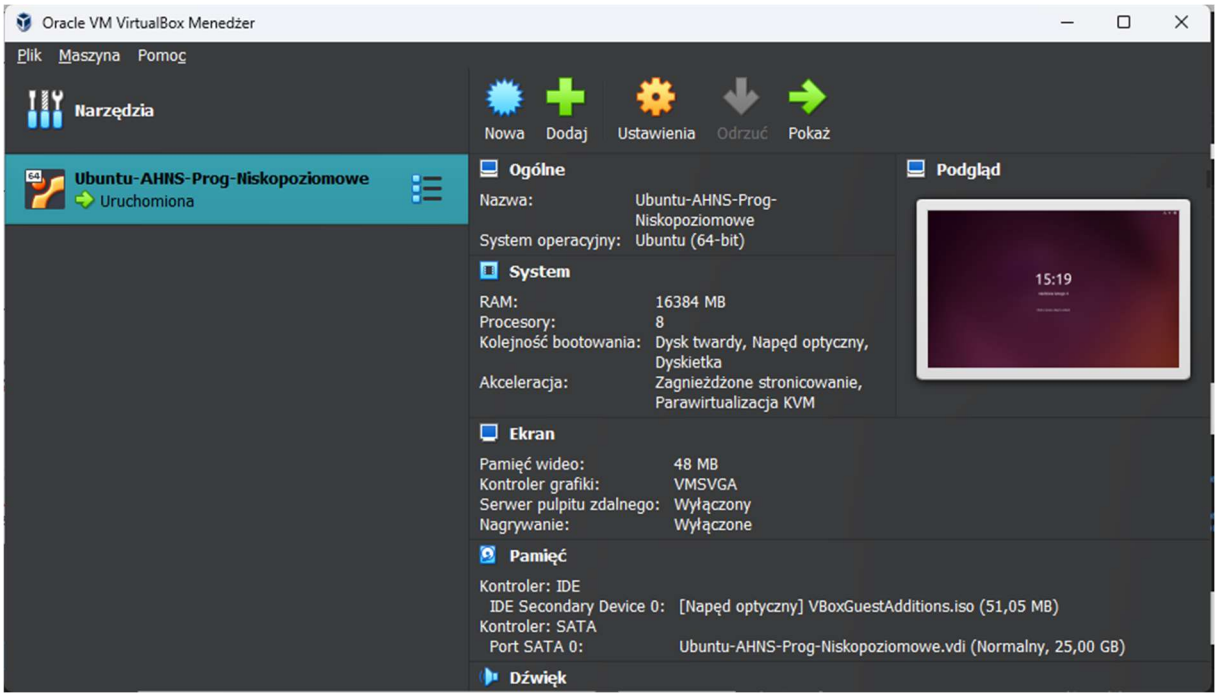
RADOM ACADEMY OF ECONOMICS

INSTRUKCJA LABORATORYJNA NUMER 1	
Przedmiot: Programowanie Niskopoziomowe	Tok: 2023/2024
Cel praktyczny zajęć: Uruchomienie pierwszego programu w języku assembly x64 NASM.	

1. Uruchomienie środowiska Ubuntu

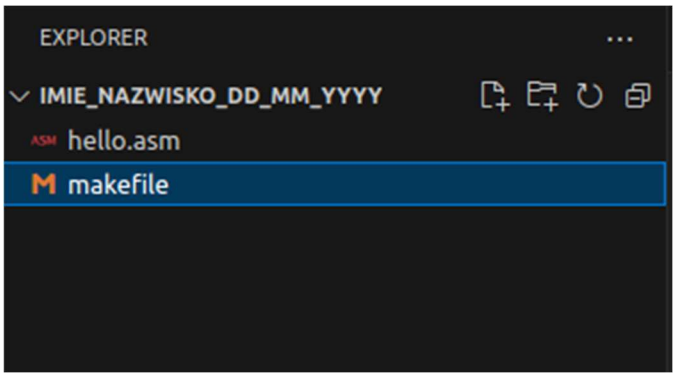
Uruchom środowisko Oracle VM VirtualBox, a następnie zaimportuj oraz uruchom obraz o nazwie: Ubuntu-AHNS-Prog-Niskopoziomowe.ova.

Użytkownik: ahnsuser
Hasło: ahns1qaz



2. Utworzenie lokalizacji dla plików z ćwiczeń

Korzystając z aplikacji Visual Studio Code załóż na pulpicie katalog prog_niskopoziomowe, a w nim kolejny o nazwie: imie_nazwisko_DD_MM_RRRR. Wewnątrz folderu utwórz dwa pliki o nazwach hello.asm oraz makefile.



3. Przygotowanie pierwszego skryptu

Do pliku hello.asm przepisz kod realizujący wyświetlenie tekstu „witaj, świecie”, z kolei do do makefile polecania służące do zbudowania programu.

hello.asm

```
section .data
    msg db      "witaj, świecie",0
section .bss
section .text
    global main
main:
    mov     rax, 1
    mov     rdi, 1
    mov     rsi, msg
    mov     rdx, 15
    syscall
    mov     rax, 60
    mov     rdi, 0
    syscall
```

makefile

```
hello: hello.o
    gcc -o hello hello.o -no-pie
hello.o: hello.asm
    nasm -f elf64 -g -F dwarf hello.asm -l hello.lst
```

4. Uruchomienie pierwszego programu

Uruchom terminal, a następnie będąc w odpowiedniej lokalizacji wykonaj kolejno polecania *make* oraz *./hello*.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash

ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$ make
nasm -f elf64 -g -F dwarf hello.asm -l hello.lst
gcc -o hello hello.o -no-pie
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$ ./hello
witaj, świecieahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY
$
```

Wynik wykonywania programu powinien być zbliżony do przedstawionego na powyższym zrzucie ekranu. Ponadto w wyniku uruchomienia poleceń z pliku makefile wygenerowane zostały pliki hello, hello.lst oraz hello.o.

```
EXPLORER
IMIE_NAZWISKO_DD_MM_YYYY
  hello
  hello.asm
  hello.lst
  hello.o
  makefile
```

Przejdź do pliku hello.lst, przyjrzyj się kolumnom znajdującym się po lewej stronie, w pierwszej z nich znajdują się względne adresy, reprezentujące lokacje w pamięci. W kolejnej kod przetłumaczony na język maszynowy w notacji szesnastkowej. Zwróć uwagę na kolumnę z przekonwertowanym kodem assemblera, w linii drugiej znajduje się na przykład konwersja łańcucha msg na szesnastkowy kod ASCII.

```
hello.asm   hello.lst
hello.lst
1                                   section .data
2  00000000 776974616A2C20C59B-         msg   db      "witaj, świecie",0
2  00000009 77696563696500
3                                   section .bss
4                                   section .text
5                                       global main
6                                   main:
7  00000000 B801000000                  mov     rax, 1
8  00000005 BF01000000                  mov     rdi, 1
9  0000000A 48BE-                       mov     rsi, msg
9  0000000C [0000000000000000]
10 00000014 BA0F000000                  mov     rdx, 15
11 00000019 0F05                        syscall
12 0000001B B83C000000                  mov     rax, 60
13 00000020 BF00000000                  mov     rdi, 0
14 00000025 0F05                        syscall
15
```

5. Dodanie znaku nowej linii

Zauważ, że w wyniku wykonywania programu wypisany w konsoli tekst „witaj, świecie” pojawił się w tej samej linii co element konsoli wskazujący na użytkownika oraz aktualną lokalizację. Ta sytuacja jest nieprawidłowa, w ramach kolejnego zadania zmodyfikuj kod w taki sposób, aby wypisany tekst oraz prompt pojawiły się w osobnych liniach, tak jak na zrzucie poniżej.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$ ./hello
witaj, świecie
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$
```

6. Przyjęcie imienia jako parametr w programie

W kolejnym kroku wprowadź do kodu modyfikację, która pozwoli na wprowadzenie parametru przez użytkownika. Tym parametrem będzie imię, które w momencie uruchomienia programu zostanie wypisane przez program, w sposób przedstawiony na zrzucie ekranu poniżej.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$ ./hello
Bartek
Witaj, Bartek

ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$
```

7. Rejestry

W języku assemblera, rejestry są małymi obszarami pamięci znajdującymi się bezpośrednio w jednostce centralnej procesora (CPU). Służą one do przechowywania danych tymczasowych i są używane do wykonywania operacji arytmetycznych, logicznych, oraz przenoszenia danych. Możesz myśleć o rejestrach jako o „pojemnikach” używanych przez procesor do przechowywania danych, instrukcji lub adresów pamięci. Dodatkowo dostęp do rejestrów jest znacznie szybszy niż do standardowych obszarów pamięci. Poniżej zamieszczona została tabela zawierająca dostępne rejestry, które będą potrzebne do dalszej pracy.

Nazwa rejestru (64bity)	Nazwa rejestru (32bity)	Opis
rax	eax	Akumulator danych
rbx	ebx	Rejestr bazowy
rcx	ecx	Rejestr licznika
rdx	edx	Rejestr danych
rsi	esi	Rejestr wskaźnikowy źródła
rdi	edi	Rejestr wskaźnikowy celu
rbp	ebp	Rejestr wskaźnikowy stosu
rsp	esp	Wskaźnik stosu
r8 - r15	r8 - r15	Rejestry ogólnego przeznaczenia (64-bitowe)
rflags	rflags	Rejestr flagi
cs, ds, es, fs, gs, ss	cs, ds, es, fs, gs, ss	Rejestry segmentowe

8. Debugowanie z użyciem GDB

Debugowanie programu jest bardzo ważnym elementem programowania, w przypadku asseblera podstawowym debugerem w systemie Linux jest GDB, działający w wierszu poleceń. Korzystanie z takiego narzędzia nie zawsze jest komfortowe, spełnia ono jednak nasze wymagania i warto je znać. Wróć do kodu programu z punktu 3, a następnie uruchom polecenie: *gdb hello*, tym momencie GDB wczytało plik wykonywalny do pamięci, efekt uruchomienia powinien być podobny do zrzutu poniżej.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   gdb - sandbox

ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY/sandbox$ gdb hello
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from hello...
(gdb)
```

Za pomocą polecenia list, możesz przewinąć kilka wierszy swojego programu, z kolei wpisując list 1 – z numerem wiersza, wyświetlony zostanie konkrety wiersz Twojego kodu. Do uruchomienia programu służy polecenie run, z kolei do zakończenia działania programu służy komenda quit. Wypróbuj działanie każdego z poleceń: *list*, *list* [z numerem wiersza], *run*, *quit* oraz *info register*, które wyświetla obecny stan rejestrów.

Przejdziemy teraz do faktycznego debugowania kodu, będąc w konsoli gdb wykonaj polecenie: *set disassembly-flavor intel* – pozwoli ono na zmianę formatu debugowanego na bardziej czytelne. Następnie uruchom polecenie *disassemble main*, rezultat powinien być zbliżony do poniższego:

```
hello.asm
sandbox > hello.asm
4   section .text
5       global main
6   main:
7       mov     rax, 1
8       mov     rdi, 1
9       mov     rsi, msg
10      mov     rdx, 15
11      syscall
12      mov     rax, 60
13      mov     rdi, 0
14      syscall
15

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   gdb - sandbox

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from hello...
(gdb) set disassembly-flavor intel
(gdb) disassemble main
Dump of assembler code for function main:
   0x0000000000401110 <+0>:     mov    eax,0x1
   0x0000000000401115 <+5>:     mov    edi,0x1
   0x000000000040111a <+10>:    movabs rsi,0x404028
   0x0000000000401124 <+20>:    mov    edx,0xf
   0x0000000000401129 <+25>:    syscall
   0x000000000040112b <+27>:    mov    eax,0x3c
   0x0000000000401130 <+32>:    mov    edi,0x0
   0x0000000000401135 <+37>:    syscall
End of assembler dump.
(gdb)
Ln 1, Col 1   Spaces: 4   UTF-8   LF   ARM
```

Jak możesz zauważyć po lewej stronie znajdują się adresy pamięci, z kolei po prawej kod naszego programu, zgodnie z wynikiem polecenia każda z operacji zajmuje 5 bajtów pamięci. Ponadto większość z użytych rejestrów nie pokrywa się z wynikiem debugowania, assembler oszacował nadmiarowe zastosowanie rejestrów 64 bitów i zastąpił je 32 bitowymi w celu oszczędzenia pamięci (rax => eax). W celu przejścia do konkretnej linii kodu możesz wykonać polecenie: *x/s 0x401115*.

9. Określanie długości zmiennej

W ramach kolejnego ćwiczenia w pierwszej kolejności załóż nowy folder o nazwie numbers wraz z plikiem numbers.asm oraz makefile, następnie przepisz poniższy kod, sprawdzając czy program uruchamia się poprawnie. Celem ćwiczenia jest poznanie sposobu określania długości zmiennej.

numbers.asm

lab1c4_pkt9 > numbers.asm

```
1   section .data
2       msg1    db  "Witaj, świecie!",10,0  ; łańcuch ze znakiem NL i 0
3       msg1Len equ $-msg1-1    ; mierzymy długość, odejmując końcowe 0
4       msg2    db  "Żyję i mam się dobrze!",10,0   ; łańcuch ze znakiem NL i 0
5       msg2Len equ $-msg2-1    ; mierzymy długość, odejmując końcowe 0
6       radius  dq  357         ; to nie łańcuch, nie można wyświetlić?
7       pi      dq  3.14        ; to nie łańcuch, nie można wyświetlić?
8   section .bss
9   section .text
10      global main
11  main:
12      push    rbp             ; prolog funkcji
13      mov     rbp,rsp         ; prolog funkcji
14      mov     rax, 1          ; 1 = wypisz
15      mov     rdi, 1          ; 1 = na stdout
16      mov     rsi, msg1       ; łańcuch do wyświetlenia
17      mov     rdx, msg1Len    ; długość łańcucha
18      syscall                 ; wyświetlamy łańcuch
19      mov     rax, 1          ; 1 = wypisz
20      mov     rdi, 1          ; 1 = na stdout
21      mov     rsi, msg2       ; łańcuch do wyświetlenia
22      mov     rdx, msg2Len    ; długość łańcucha
23      syscall                 ; wyświetlamy łańcuch
24      mov     rsp,rbp         ; epilog funkcji
25      pop     rbp             ; epilog funkcji
26      mov     rax, 60         ; 60 = wyjście
27      mov     rdi, 0          ; 0 = kod wyjściowy sygnalizujący sukces
28      syscall                 ; zakończ
29
```

makefile

lab1c4_pkt9 > makefile

```
1  # plik makefile dla numbers.asm
2  numbers: numbers.o
3      gcc -o numbers numbers.o -ggdb -no-pie
4  numbers.o: numbers.asm
5      nasm -f elf64 -g -F dwarf numbers.asm -l numbers.lst
6
```

Linia 3 oraz 5 o treści *msg1Len equ $-msg1-1* pozwala na określenie długości zmiennej, w następujący sposób: *$* – weź obecną lokację pamięci, *-msg1* – odejmij od niej lokację pamięci, *-1* – odejmij zero na końcu łańcucha, wynik tej operacji zapisywany jest do zmiennej msg1Len. W kodzie został umieszczony kod prologu oraz epilogu, co jest wymagane w tym przypadku do prawidłowego funkcjonowania GDB. Ponownie uruchom debugger poleceniami: gdb numbers, disassemble main. Spróbuj w GDB wyświetlić zawartość poszczególnych zmiennych w następujący sposób: *x/s &msg1,*

zastanów się w jaki sposób wyświetlić zawartości pozostałych msg1Len, radius oraz pi. Czy dla wszystkich z tych zmiennych możemy sprawdzić zawartość z poziomu konsoli GDB?

```
(gdb) x/s &msg1
0x404028 <msg1>:        "Witaj, świecie!\n"
(gdb)
```

10. Wypisywanie zmiennych liczbowych

Do tej pory wszystkie zmienne, które wypisywaliśmy na ekran były łańcuchami znaków, należy zastanowić się jak możemy wyświetlić zmienne radius oraz pi z poprzedniego przykładu. Jednym z rozwiązań mogłoby być ich przekształcenie na łańcuchy, a następnie wydrukowanie w standardowy sposób, nie jest to jednak operacja tak prosta jak się wydaje. Dlatego, też w celu rozwiązania tego problemu wykorzystamy funkcję printf z języka C, wyświetlającą dane na ekran.

W tym celu zmodyfikuj kod z poprzedniego zadania dodając w pierwszej linii informację mówiącą o deklaracji zewnętrznej funkcji w następujący sposób: *extern printf.* Następnie zmień wywołanie syscall na *call printf,* zadeklaruj również nową zmienną o następującej treści: *strToPrint db „Zawartosc zmiennej msg1: %s”,10,0.* Wprowadź zmiany w rejestrach zgodnie ze zrzutem poniżej, a następnie uruchom program i sprawdź czy rezultat jest zgodny z oczekiwaniami.

numbers.asm

lab1c4_pkt10 > numbers.asm

```
extern printf
section .data
    msg1        db      "Witaj, świecie!",0
    radius      dq      357
    pi          dq      3.14
    strToPrint  db      "Zawartosc zmiennej msg1: %s", 10,0;
section .bss
section .text
    global main
main:
    push    rbp
    mov     rbp, rsp
    mov     rdi, strToPrint
    mov     rsi, msg1
    mov     rax, 0
    call    printf
    mov     rsp,rbp
    pop     rbp
    mov     rax, 60
    mov     rdi, 0
    syscall
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - lab1c4_pkt10

```
Zawartosc zmiennej msg1: Witaj, świecie!
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY/lab
1c4_pkt10$
```

Spróbuj w podobny sposób zawartości zmiennych radius oraz pi, najprawdopodobniej się to nie uda. Znajdź w internecie dokumentację do funkcji printf, a następnie dostosuj kod, tak aby rezultat pokrywał się z poniższym:

```
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY/lab1c4_pkt10$ ./numbers
Zawartosc zmiennej msg1: Witaj, świecie!
Zawartosc zmiennej radius: 357
Zawartosc zmiennej pi: 3.140000
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY/lab1c4_pkt10$
```

11. Utworzenie aplikacji kalkulator liczb całkowitych

Stwórz kalkulator liczb całkowitych, który będzie spełniać następujące wymagania:

- Użytkownik ma możliwość wybrania jednej z czterech dostępnych operacji: dodawanie, odejmowanie, mnożenie, dzielenie
- Wartości przekazane do obliczenia przekazywane są przez użytkownika
- Wynik operacji jest wypisywany na ekran